# SIVF: GPU-Resident IVF Index for Streaming Vector Analytics

Anonymous Author(s)

## Abstract

GPU-accelerated Inverted File (IVF) index is one of industry standards for large-scale vector analytics but relies on static VRAM layouts that hinder real-time mutability. Our benchmark and analysis reveal that existing designs of GPU IVF necessitate expensive CPU-GPU data transfers for index updates, causing system latency (at the granularity of 10K 128-dimensional vectors) to spike from milliseconds to seconds in streaming scenarios. We present SIVF, a GPU-native index that enables high-velocity, in-place mutation via a series of new data structures and algorithms, such as conflict-free slab allocation and coalesced search on non-contiguous memory. SIVF has been implemented and integrated into the open-source vector search library, Faiss. Evaluation against 6 baselines (GPU IVF, GPU CAGRA, GPU Flat, HNSW, NSG, LSH) with 5 vector datasets (synthetic, Deep1B, SIFT1M, T2I-1B, GIST1M) suggests that SIVF effectively removes the I/O bottleneck of GPU indices for streaming vectors, therefore transforming vector search from a static retrieval task into an integral component of real-time vector data analytics.

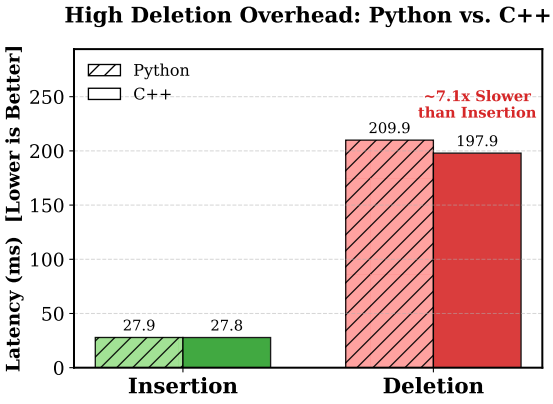


**Figure 1: The asymmetry between insertion and deletion.**

## 1 Introduction

Real-world vector analytics applications, ranging from search engines to dynamic retrieval-augmented generation (RAG) of large language models (LLMs), increasingly operate on streaming data where timeliness is critical [29, 37]. These systems necessitate a sliding-window model where expired vectors must be invalidated promptly as new vectors arrive to maintain bounded memory footprint. However, existing GPU-accelerated approximate nearest neighbor (ANN) indices are predominantly optimized for static, write-once-read-many workloads [32, 54].

To quantify the gap between insertion and eviction performance, we conducted benchmarks with the industry-standard Faiss library [20] on an NVIDIA RTX 6000 GPU hosted at the Chameleon Cloud [23]. We utilized the SIFT1M dataset [22] (128-dimensional) to simulate a realistic sliding-window scenario. Specifically, we initialized a standard `IVF1024` index pre-populated with 100,000 vectors and measured the wall-clock latency of ingesting a batch of 10,000 new vectors and subsequently evicting the oldest 10,000 vectors. As illustrated in Figure 1, we observe a drastic performance asymmetry between insertion and deletion performance. While GPU parallelism accelerates the insertion of the 10K-vector batch to approximately 28 ms, the eviction process for the same batch size is significantly slower, spiking the latency to ~200 ms. This high latency is observed in the native C++ implementation, which performs nearly identically to the Python binding. The negligible difference between the compiled C++ core and the interpreted Python interface suggests that the bottleneck is not an implementation detail: If the issue were merely inefficient code, the C++ implementation would be expected to significantly outperform Python. The lack of such divergence suggests that both are bound by the same architectural limitation: the absence of a GPU-resident deletion operator, which forces expensive CPU-GPU roundtrips for eviction.

Our analysis of the Faiss source code [6] further reveals that this performance asymmetry stems from a rigid class hierarchy that enforces a fallback to host-side processing on CPUs. In the library's architecture, the data eviction interface is defined in the abstract base class `faiss::Index` via the `remove_ids` virtual method. While CPU-based implementations override this method to provide efficient in-memory deletion logic (e.g., `memmove`), the GPU counterparts inherit the base implementation which lacks native support. Consequently, current GPU indices rely on a *CPU-GPU Roundtrip* pattern for eviction. That is, to delete even a single vector, the system must transfer the entire index state from VRAM to host memory, perform the compaction on the CPU, and re-upload the modified index to the device. This heavy I/O burden saturates the PCIe bus and prevents the system from utilizing the GPU's compute throughput, capping the maximum sustainable ingestion rate in streaming scenarios.

The absence of a GPU-resident eviction operator is not an oversight but a consequence of the complexity involved in managing dynamic data structures within VRAM. Unlike CPU indices that utilize dynamic arrays, GPU indices rely on pre-allocated, contiguous memory blocks to maximize memory coalescing and bandwidth utilization. Supporting in-place deletion on GPUs presents at least 3 challenges. First, maintaining contiguous arrays requires dynamic memory management. Frequent reallocation and compaction of inverted lists within VRAM lead to severe memory fragmentation and costly data movement. Second, concurrent modification requires complex synchronization. Managing fine-grained locks to prevent race conditions when thousands of concurrent GPU threads attempt to modify shared list structures introduces significant latency. Third, standard inverted file (IVF) indices lack an ID-to-Location mapping. Without a reverse index, locating a specific ID for deletion requires scanning all inverted lists, an operation with $O(N)$ complexity that negates the benefits of GPU acceleration. These constraints make naive GPU eviction prohibitively inefficient.

To resolve these limitations, we propose SIVF, a GPU-resident indexing method that supports high-throughput mutability through four specialized device-side algorithms. First, to enable nonblocking writes, we design a *Lock-Free Ingestion* protocol (Algorithms 1 & 2) that manages a pool of fixed-size slabs. This protocol utilizes `atomic Compare-And-Swap` for slot reservation and speculative head updates, allowing concurrent threads to append to linked lists without global locks. Second, to guarantee data consistency under CUDA's relaxed memory model, we apply a strict publication ordering; the kernel issues device-wide memory fences (`__threadfence()`) before committing validity bits, ensuring readers never observe partially initialized payloads. Third, to mitigate the latency of pointer chasing, we propose *Warp-Cooperative Search* (Algorithm 3). By aligning the slab capacity ($C = 32$) with the hardware warp width, this algorithm enables threads to cooperatively load and evaluate non-contiguous memory blocks while preserving coalesced access patterns. Finally, we address deletion by maintaining a GPU-resident Address Translation Table along with lazy eviction (Algorithm 4), allowing SIVF to decouple logical removal from physical data movement, which makes eviction to a constant-time operation.

SIVF has been implemented and integrated into Faiss [6] as a new index through the `GpuIndexSIVF` class. Our implementation leverages CUDA for device-side kernels and employs a dual-view memory management strategy to maintain the consistency between host and device states. All new algorithms as well as key data structures are encapsulated within a `SlabManager` class that abstracts the complexity of dynamic memory operations on the GPU.

We have evaluated SIVF against a comprehensive set of baselines within Faiss, including CAGRA, IVF, Flat, HNSW, NSG, and LSH indices. Our evaluation was carried out on five datasets representing a variety of modalities, dimensions, and scalability: synthetic, Deep1B, SIFT1M, T2I-1B, and GIST1M. Experimental results show that SIVF achieves up to $13,307\times$ reduction in deletion latency (Fig. 8) and improves ingestion throughput by $36\times$–$120\times$ compared to existing GPU indices (Fig. 7). In end-to-end sliding window scenarios, SIVF delivers a $163\times$–$262\times$ speedup (Fig. 10) with a negligible memory overhead less than 0.8% (Fig. 11), effectively closing the gap between static and streaming vector analytics performance.

In summary, this paper makes the following contributions:

- We identify the CPU-GPU bottleneck in existing GPU vector indices when being used for sliding-window scenarios, demonstrating that the lack of native deletion support renders them unsuitable for streaming vector analytics.
- We propose a new indexing method, namely SIVF, that synthesizes a series of GPU-optimized data structures and algorithms for streaming vector analytics. This design enables $O(1)$ time complexity for deletion and constant-time insertion overhead with negligible memory overhead.
- We implement SIVF in the state-of-the-art vector search library Faiss and evaluate SIVF against a broad spectrum of baselines (GPU CAGRA, GPU Flat, GPU IVF, HNSW, NSG, and LSH) on 5 datasets: synthetic, Deep1B, SIFT1M, T2I-1B, and GIST1M. The results demonstrate that SIVF achieves up to $13,307\times$ reduction in deletion latency, $120\times$ faster ingestion, and $266\times$ end-to-end sliding window speedup with less than 0.8% memory overhead.

## 2 Related Work

Approximate Nearest Neighbor (ANN) search has been extensively studied, with a rich body of work spanning CPU-based dynamic indices, GPU-accelerated static indices, and hybrid approaches that integrate attribute filtering or quantization. This section reviews these developments, highlighting their respective strengths and limitations in the context of streaming vector analytics.

### 2.1 Optimization of Graph-Based Indices

Graph-based indices remain the state-of-the-art for high-recall retrieval. To address performance bottlenecks in production, frameworks like VSAG [55] optimize memory access patterns and parameter tuning, while SOAR [43] improves indexing structures. Of note, CAGRA [38] establishes a new state-of-the-art for static graph construction and search throughput on GPUs; however, it is fundamentally designed for write-once-read-many workloads and lacks native support for the efficient, fine-grained in-place deletions required by streaming applications. Navigational efficiency is a key optimization target; SHG [11] introduces a hierarchical graph with shortcuts to bypass redundant levels, and probabilistic routing methods [33] have been proposed to enhance traversal. Several works focus on distance metric optimizations: ADA-NNS [21] utilizes angular distance guidance to filter irrelevant neighbors, DADE [5] accelerates comparisons via data-aware distance estimation in lower dimensions, and HSCG [41] adapts the Monotonic Relative Neighbor Graph for cosine similarity using hemi-sphere centroids. Efforts to improve graph construction and maintenance include LIGS [3], which employs locality-sensitive hashing (LSH) to simulate proximity graphs for faster updates, and CSPG [51], which crosses sparse proximity graphs. Addressing robustness, Hua et al. [13] propose dynamically detecting and fixing graph hardness for out-of-distribution queries. MIRAGE-ANNS [45] attempts to bridge the gap between incremental and refinement-based construction. Furthermore, theoretical frameworks like Subspace Collision [48] provide guarantees on result quality using clustering-based indexing.

Collectively, while these approaches maximize navigational efficiency for static data, they fundamentally rely on complex edge dependencies that are prohibitively expensive to maintain on GPUs. Unlike SIVF, they lack the architectural support for $O(1)$ lock-free mutation, rendering them unsuitable for high-churn streaming workloads where sub-millisecond update latency is critical.

### 2.2 Attribute-Filtered and Hybrid Search

The integration of structured constraints with vector analytics has led to a surge in Filtered ANN research. Li et al. [26] provide a comprehensive taxonomy and benchmark of these methods. A primary focus is handling dynamic updates and specific filter types such as ranges or windows. For range filtering, UNIFY [30] introduces a segmented inclusive graph to support pre-, post-, and hybrid filtering strategies. Dynamic environments are addressed by DIGRA [19], which uses a multi-way tree structure for dynamic graph indexing, and RangePQ [53], which offers a linear-space indexing scheme for range-filtered updates. Peng et al. [39] propose dynamic segment graphs to handle mixed streams of data and queries. Regarding specific constraints, Wang et al. [46] present a robust framework

for general attribute constraints. Engels et al. [7] focus on window filters, while WoW [47] develops a window-graph based index to handle window-to-window incremental construction.

However, these contributions primarily optimize algorithmic filtering logic rather than the underlying storage architecture. They remain constrained by the static memory layouts of standard GPU indices, whereas SIVF fundamentally redesigns the VRAM management to enable the high-throughput, in-place mutability that is a prerequisite for such dynamic operations.

## 2.3 Quantization and Scalability

To manage high-dimensional data at scale, quantization and system-level optimizations are critical. RaBitQ [10] and its extended version [9] provide theoretical error bounds for bitwise quantization with flexible compression rates. SymphonyQG [12] integrates quantization codes directly with graph structures to reduce memory access overhead, while LoRANN [18] utilizes low-rank matrix factorization for compression. In terms of system architecture, Tagore [28] leverages GPUs for accelerating graph index construction. For distributed environments, HARMONY [49] employs a multi-granularity partition strategy to balance load. WebANNS [31] enables efficient search within web browsers via WebAssembly. From a theoretical perspective, Indyk and Xu [17] analyze the worst-case performance limits of popular implementations. Finally, DARTH [1] introduces declarative recall targets through adaptive early termination to balance performance and result quality.

While these techniques reduce storage footprints or distribute load, they predominantly operate on static memory layouts that are brittle under frequent updates. SIVF addresses a complementary challenge: it provides the dynamic memory substrate that allows these scalable architectures to support real-time streams without succumbing to VRAM fragmentation or locking overheads.

*GPU Acceleration Techniques.* Recent HPC work highlights how careful kernel and data-layout design can unlock GPU performance across data-intensive workloads. FZ-GPU demonstrates a fully parallel compression pipeline with both warp-aware bitwise operations and shared-memory efficiency [52]. For sparse linear algebra, SpMV designs reduce preprocessing overhead while improving load balance and memory access locality [2], and GPU-aware preconditioning further emphasizes locality and coalescence as first-order concerns on modern GPU architectures [24]. Beyond compute kernels, GPU-enabled asynchronous checkpoint caching and prefetching treat GPU memory as a first-class tier to improve end-to-end throughput for I/O-heavy workflows [35]. Format choices and composition are also critical for irregular workloads, as shown by automatic sparse format composition for SpMM that avoids expensive autotuning while delivering strong performance [40]. Another line of work focuses on reliability, debugging, and cluster-scale GPU management, which together shape how GPU-centric systems are built and operated. GPU-FPX provides low-overhead floating-point exception tracking for NVIDIA GPUs [27], FPBOXer improves scalable input generation for triggering floating-point exceptions in GPU programs [44], and FloatGuard extends whole-program exception detection to AMD GPUs and highlights cross-vendor portability concerns [36]. At the application and cluster level, SIMCoV-GPU studies multi-node, multi-GPU acceleration for irregular agent-based simulations [25], FASOP automates fast search for near-optimal transformer parallelization on heterogeneous GPU clusters [16], and serverless platforms motivate GPU sharing and scheduling mechanisms such as ESG and FluidFaaS [14, 15]. These works reinforce the importance of aligning work granularity with warps, minimizing synchronization overhead, and designing memory layouts that preserve locality, which are relevant to GPU-resident indexing under dynamic updates.

However, applying these general-purpose HPC optimizations to vector search remains non-trivial due to the irregular memory access patterns of IVF traversal. SIVF bridges this gap by tailoring these micro-architectural principles, specifically warp-aligned slabs and lock-free synchronization, to construct the first GPU-resident index capable of sustaining high-velocity updates without compromising retrieval integrity.

# 3 Design and Implementation of SIVF

This section presents the design of SIVF for high-concurrency updates on GPUs. We first introduce the Slab-based Dynamic Memory Allocator (Section 3.1), which represents each IVF list as a linked chain of fixed-capacity slabs with per-list head pointers $H[\cdot]$ and per-slab metadata (`next`, `valid_count`, `validity_bitmap`) (Algorithm 1). Building on SDMA, we describe the parallel ingestion kernel (Section 3.2) in which each thread inserts one vector by reserving a slot via `atomicCAS` on `valid_count` and publishing visibility by setting a validity bit after `__threadfence()` (Algorithm 2). We then present a warp-cooperative search kernel (Section 3.3) that assigns one warp to one query and evaluates one slab per traversal step by consulting the validity bitmap before reading payloads (Algorithm 3). Finally, we detail lazy eviction (Section 3.4), which performs constant-time deletion by looking up coordinates in the address table $\mathcal{T}$ and atomically clearing the corresponding bitmap bit (Algorithm 4).

## 3.1 Slab-based Memory Management

We propose a Slab-based Dynamic Memory Allocator (SDMA) that replaces monolithic, variable-length inverted lists with linked chains of fixed-size blocks on GPU. SDMA pre-allocates a contiguous slab pool; each slab stores up to $C$ vectors and a small metadata header $M = \langle n_{next}, b_{valid}, c_{valid} \rangle$, where $n_{next}$ is the next-slab pointer, $b_{valid}$ is a $C$-bit validity bitmap, and $c_{valid}$ tracks occupancy. We set $C = 32$ to match the warp size. Each IVF list $\ell$ is represented by a head pointer $H[\ell]$ to the first slab in its chain. Allocation draws a fresh slab id from a global stack pointer $P_{top}$ via $s = \texttt{atomicSub}(P_{top}, 1) - 1$, and links the new slab by setting its $n_{next}$ to the previous head and then publishing $H[\ell]$ with an atomic compare-and-swap. Insertion for a vector id $v_{id}$ first reads the current head slab $s \leftarrow H[\ell]$, reserves a free slot $o$ in $s$ by atomically updating $c_{valid}$, writes the payload into `slab data[`$s$`][`$o$`]`, and publishes visibility by atomically setting bit $o$ in $b_{valid}$. To support $O(1)$ deletion, SDMA maintains an address table $\mathcal{T}$ that records the physical coordinate of each id, $\mathcal{T}(v_{id}) = \langle s, o \rangle$; deletion performs a direct lookup and atomically clears bit $o$ in $b_{valid}$, then marks $\mathcal{T}(v_{id})$ invalid. Algorithm 1 summarizes these core operations.

**Algorithm 1** Core operations of SDMA

**Global State:** slab pool metadata and data, per-list head pointers $H[\cdot]$, stack pointer $P_{top}$, address table $\mathcal{T}$
**Input:** vector identifier $v_{id}$, vector $x$, list assignment $\ell$
**procedure** INSERT($v_{id}, x, \ell$)
  $s \leftarrow H[\ell]$
  **while** $s$ is invalid **or** no free slot exists in slab $s$ **do**
    $s_{new} \leftarrow$ atomicSub($P_{top}, 1$) $- 1$
    Initialize metadata $s_{new}$ with $b_{valid} = 0$, $n_{next} = s$
    Update $H[\ell]$ to $s_{new}$ if unchanged, otherwise retry
    $s \leftarrow H[\ell]$
  **end while**
  Reserve a free slot $o$ in slab $s$ via atomic update of $c_{valid}$
  Write $x$ to slab data$[s][o]$
  Atomically set bit $o$ in $b_{valid}$
  $\mathcal{T}(v_{id}) \leftarrow \langle s, o \rangle$
**end procedure**
**procedure** DELETE($v_{id}$)
  $\langle s, o \rangle \leftarrow \mathcal{T}(v_{id})$
  **if** $\langle s, o \rangle$ is valid **then**
    Atomically clear bit $o$ in $b_{valid}$ of slab $s$
    Mark $\mathcal{T}(v_{id})$ as invalid
  **end if**
**end procedure**

## 3.2 Lock-free Parallel Ingestion

SIVF uses a fully parallel ingestion kernel in which each CUDA thread inserts one vector into its assigned IVF list $\ell$. Each list is a singly linked chain of fixed-capacity slabs, with head pointer $H[\ell]$ and per-slab metadata `valid_count`, `validity_bitmap`, and `next`. Insertion follows a reserve, write, publish protocol. A thread first reads the current head slab $h = H[\ell]$ and attempts to reserve a slot by incrementing `slab_meta[`$h$`].valid_count` using `atomicCAS`; the reserved slot index is the old counter value $c$. On success, the thread writes the payload to `slab data[`$h$`][`$c$`]` and the id to `slab_ids[`$h$`][`$c$`]`, updates the address table $\mathcal{T}(v_{id}) \leftarrow \langle h, c \rangle$, then executes `__threadfence()` and sets the corresponding validity bit using `atomicOr`. The validity bit is the only publication signal read by search, so this ordering ensures that a reader observing the bit set never sees partially initialized payload or a missing table entry. If the head slab is absent or full, the thread allocates a new slab id from the global pool via `atomicSub` on $P_{top}$, initializes its metadata (`valid_count`, `validity_bitmap`, `next`), executes `__threadfence()`, and attempts to publish it as the new head with `atomicCAS(&H[`$\ell$`], h, s_new)`. If head publication fails under contention, the thread retries without reclaiming the allocated slab, avoiding allocator-side synchronization in the hot path. The full per-thread protocol is given in Algorithm 2.

## 3.3 Coalesced Search on Non-Contiguous Slabs

SIVF stores each inverted list as a linked chain of fixed-capacity slabs, which replaces contiguous scans with traversal via `next`. To retain high GPU throughput, we use a warp-cooperative search kernel that matches slab capacity to the warp width ($C = 32$). The kernel launches one warp per query. The warp first stages the query vector into shared memory, then probes *nprobe* coarse lists returned by the quantizer. For each probed list $\ell$, the warp traverses the slab chain starting from $s = H[\ell]$. At each slab $s$, lane $j$ consults `slab_meta[`$s$`].validity_bitmap` and evaluates slot $j$ only if the corresponding bit is set. If set, the lane loads the payload from `slab data[`$s$`][`$j$`]`, retrieves the label from `slab_ids[`$s$`][`$j$`]`, computes the distance to the shared query, and updates a small per-lane top-$k$ structure kept in registers. After traversal, lanes write their local top-$k$ candidates to shared memory and one lane merges them into the final top-$k$ result for the query. To ensure termination under unexpected corruption, traversal is bounded and the kernel breaks on self-loops (`md.next == s`). Algorithm 3 summarizes the procedure.

**Algorithm 2** Lock-free ingestion protocol in SIVF

Input: vector $x$, identifier $v_{id}$, list assignment $\ell$
Global: head array $H[\cdot]$, slab metadata, slab data, free list, stack pointer $P_{top}$, Address Table $\mathcal{T}$
$attempts \leftarrow 0$
**while** $attempts <$ MAX_INSERT_ATTEMPTS **do**
  $attempts \leftarrow attempts + 1$
  $h \leftarrow H[\ell]$
  **if** $h \neq -1$ **then**
    $c \leftarrow$ slab_meta$[h]$.valid_count
    **if** $c < C$ **then**
      **if** atomicCAS(&slab$[h]$.valid_cnt) $== c$ **then**
        Write payload $x$ to slab data$[h][c]$
        Write identifier to slab id buffer$[h][c]$
        $\mathcal{T}(v_{id}) \leftarrow \langle h, c \rangle$
        __threadfence()
        atomicOr(&slab_meta[h].validity_bitmap)
        **return**
      **end if**
    **end if**
  **end if**
  $t \leftarrow$ atomicSub($P_{top}, 1$)
  **if** $t \leq 0$ **then**
    atomicAdd($P_{top}, 1$)
    **return**
  **end if**
  $s_{new} \leftarrow$ free_list$[t - 1]$
  Set slab_meta$[s_{new}]$.valid_count $\leftarrow 1$
  Set slab_meta$[s_{new}]$.validity_bitmap $\leftarrow 0$
  Set slab_meta$[s_{new}]$.next $\leftarrow h$
  __threadfence()
  **if** atomicCAS$(\&H[\ell], h, s_{new}) == h$ **then**
    Write payload $x$ to slab data$[s_{new}][0]$
    Write identifier to slab id buffer$[s_{new}][0]$
    $\mathcal{T}(v_{id}) \leftarrow \langle s_{new}, 0 \rangle$
    __threadfence()
    atomicOr(&slab_meta[s_new].validity_bitmap)
    **return**
  **end if**
**end while**

**Algorithm 3** Warp-cooperative search in SIVF

Input: queries $Q$, probed list ids *coarse_ids*, head array $H[\cdot]$
Input: slab metadata and data, buffer *slab_ids*, $k$, *nprobe*
Output: top-$k$ distances and labels for each query
**for** each query index $q$ in parallel **do**
  Launch one warp for query $q$
  Copy $Q[q]$ into shared memory
  Initialize per-thread local top-$k$ lists in registers
  **for** $p = 0$ to $nprobe - 1$ **do**
    $\ell \leftarrow coarse\_ids[q, p]$
    **if** $\ell$ is invalid **then**
      continue
    **end if**
    $s \leftarrow H[\ell]$
    **while** $s$ is valid and traversal bound not exceeded **do**
      $md \leftarrow slab\_meta[s]$
      **if** $md.next = s$ **then**
        break
      **end if**
      **if** $j < 32$ and $md.validity\_bitmap[j]$ is set **then**
        Load vector $\mathbf{x}_{s,j}$ from slab data
        Compute $d(\mathbf{q}, \mathbf{x}_{s,j})$
        $id \leftarrow slab\_ids[s, j]$
        Insert $(d, id)$ into local top-$k$
      **end if**
      $s \leftarrow md.next$
    **end while**
  **end for**
  Write local top-$k$ lists to shared memory
  One thread merges 32 lists and writes final top-$k$ outputs
**end for**

## 3.4 Lazy Eviction via Address Translation Table

SIVF supports constant-time deletion by combining an Address Translation Table (ATT) with bitmap-based lazy eviction. The ATT $\mathcal{T}$ maps each identifier $u$ to its physical coordinate in the slab pool as a 64-bit value $\mathcal{T}[u] = (\text{idx}_{slab} \ll 32) \mid \text{idx}_{slot}$, or `INVALID` if absent. Given $\mathcal{T}[u]$, a deletion thread decodes $(\text{idx}_{slab}, \text{idx}_{slot})$ and clears the corresponding bit in the slab validity bitmap using an atomic read-modify-write (`atomicAnd` with $mask = \sim (1 \ll \text{idx}_{slot})$). The payload is not moved or overwritten; search consults the bitmap and skips slots whose bits are cleared, so deletion removes a vector from the search space without compaction. To handle duplicates and races among deletions, the kernel uses the pre-update bitmap value returned by `atomicAnd` to detect a $1 \to 0$ transition and performs bookkeeping only on that transition (e.g., updating counters), then writes $\mathcal{T}[u] \leftarrow$ `INVALID` so repeated deletions become no-ops. Algorithm 4 summarizes the procedure.

## 3.5 Theoretical Analysis

*3.5.1 Correctness.* We prove three properties. The first establishes the correctness of parallel ingestion in Algorithm 2. The second shows that search in Algorithm 3 is safe under concurrent ingestion and deletion. The third proves that lazy eviction in Algorithm 4 is linearizable with a clear linearization point.

**Algorithm 4** Lazy eviction with ATT in SIVF

Input: deletion identifiers $U = \{u_1, \ldots, u_m\}$
Global: Address Table $\mathcal{T}$, slab metadata with bitmap $\mathcal{B}$ and counter `valid_count`
**for** each $u$ in $U$ in parallel **do**
  $coord \leftarrow \mathcal{T}[u]$
  **if** $coord$ = INVALID **then**
    continue
  **end if**
  $\text{idx}_{slab} \leftarrow coord \gg 32$
  $\text{idx}_{slot} \leftarrow coord \;\&\; (2^{32} - 1)$
  $mask \leftarrow \sim (1 \ll \text{idx}_{slot})$
  $old \leftarrow$ `atomicAnd`$(\mathcal{B}_{\text{idx}_{slab}}, mask)$
  **if** $((old \gg \text{idx}_{slot}) \;\&\; 1) = 1$ **then**
    `atomicSub(&valid_count, 1)`
    `atomicAdd(&deleted_count, 1)`
    $\mathcal{T}[u] \leftarrow$ INVALID
  **end if**
**end for**

We use the same state as the pseudocode. For each list id $\ell$, $H[\ell]$ stores the head slab id. Each slab $s$ has metadata `slab_meta[`$s$`]` including `next`, `valid_count`, and a 32 bit `validity_bitmap`. A slot $(s, o)$ is logically present if and only if bit $o$ in the bitmap is 1. Payloads and ids are stored in `slab data[`$s$`][`$o$`]` and `slab_ids[`$s$`][`$o$`]`. The address table $\mathcal{T}$ maps a vector id $u$ to a coordinate $(s, o)$ encoded in `coord`, or `INVALID`.

**Theorem 3.1** (Parallel ingestion is safe and linearizable)**.** *Consider any concurrent execution of Algorithm 2. For every insertion that returns successfully, there exists a unique slot $(s, o)$ such that (i) the insertion writes the payload and id into that slot, then sets bit $o$ in* `validity_bitmap` *exactly once, (ii) once the bit is set, the slot contains a fully initialized payload and id, and (iii) the insertion can be linearized at the atomic bit set operation* `atomicOr` *that makes the slot visible.*

Proof. We first argue the uniqueness of the chosen slot for a successful insertion.

In the existing head case, the code reads

$$c = \texttt{slab\_meta}[h].\texttt{valid\_count}$$

and attempts to reserve index $c$ using an atomic compare and swap. Only one thread can succeed for a given observed $c$, because the compare and swap updates a single memory location atomically. Therefore, among concurrent contenders for the same slab $h$, at most one insertion obtains a given index $c$. In the new slab case, the thread publishes its new slab as the head using `atomicCAS(&H[`$\ell$`], h, s_new)`; only one thread can win this publication for a fixed old head value $h$, and the winner uses slot $o = 0$ in its newly allocated slab. Thus, every successful insertion selects exactly one slot $(s, o)$.

We next argue that the chosen slot is initialized before it becomes visible. In both code paths, after reserving the slot, the writer performs the payload write to `slab data[`$s$`][`$o$`]` and the id write to `slab_ids[`$s$`][`$o$`]`, then writes $\mathcal{T}(v_{id}) \leftarrow \langle s, o \rangle$, then executes `__threadfence()`, and only after the fence performs `atomicOr` to set bit $o$ in the bitmap. The intended CUDA guarantee used here is

that the fence makes the prior global writes visible before a later atomic operation performed by the same thread becomes visible to other threads. Therefore, once bit $o$ is observed as set, the payload and id written earlier by that insertion are already visible as well.

Finally, we justify linearizability of a successful insertion with linearization point at the atomic bit set. The membership predicate used throughout the design is exactly the bitmap bit. Before the `atomicOr`, the bit is 0, so the slot is logically absent. After the `atomicOr` takes effect, the bit is 1, so the slot is logically present. Since `atomicOr` is an atomic operation on the bitmap word, it takes effect at a single instant and provides a valid linearization point. At that instant, the slot becomes visible and, by the fence argument above, it is already fully initialized. □

**Theorem 3.2** (Search safety under concurrent ingestion and deletion)**.** *In any concurrent execution of Algorithms 2, 3, and 4, every payload and id read performed by Algorithm 3 is fully initialized.*

Proof. Algorithm 3 reads a slot $(s, j)$ only if it first observes that bit $j$ in `slab_meta[`$s$`].validity_bitmap` is set. When the search observes the bit set, the slot must have been published by some successful insertion. By Theorem 3.1, the publication point is the atomic bit set in Algorithm 2, and the payload and id writes occur before the fence, which occurs before the bit set. The same fence guarantee used in Theorem 3.1 implies that once the bit set is visible, the earlier payload and id writes are also visible. Therefore, the search cannot observe a partially initialized payload or id.

Deletion does not write payloads or ids. Algorithm 4 only clears the validity bit and possibly updates counters and $\mathcal{T}$. Hence, even if a search races with a delete, the only effect is whether the search sees the bit as 1 or 0. If it sees 0, it skips the slot and reads nothing. If it sees 1, it reads a payload and id that were fully initialized before publication, by the argument above. In neither case does the search read partially initialized data.

Algorithm 3 also terminates because its traversal is bounded, and it includes an explicit self loop guard (`md.next = s`) before following `next`. Concurrent insertions can add a new head slab, but they do not require the search to revisit already visited slabs within the traversal bound. □

**Theorem 3.3** (Lazy eviction is linearizable and makes deleted vectors invisible)**.** *Consider any concurrent execution of Algorithms 3 and 4. For any delete request on id $u$ such that $\mathcal{T}[u] \neq$ `INVALID` and decodes to $(idx_{slab}, idx_{slot})$, define*

$$mask = \sim (1 \ll idx_{slot}).$$

*Then the atomic operation* `old = atomicAnd(&`$\mathcal{B}_{\mathrm{idx}_{slab}}$`, mask)` *in Algorithm 4 is a linearization point for logical deletion. After this atomic operation takes effect, the slot becomes logically absent from the search space until a later insertion sets the same bit again. Repeated deletes of the same id are safe.*

Proof. If $\mathcal{T}[u]$ = `INVALID`, Algorithm 4 performs no state change and the delete can be linearized at the `INVALID` check.

Otherwise, the delete decodes `coord` into $(\mathrm{idx}_{slab}, \mathrm{idx}_{slot})$, computes $mask = \sim (1 \ll \mathrm{idx}_{slot})$, and executes

`old = atomicAnd(&`$\mathcal{B}_{\mathrm{idx}_{slab}}$`, mask)`

to clear the corresponding bit. The intended guarantee is that this atomic read modify write takes effect at a single instant on the bitmap word. We choose that instant as the linearization point.

Algorithm 3 uses the validity bit as the membership predicate and checks it before dereference. Thus, after the clear takes effect, any search that reads the bitmap and sees the bit as 0 skips the slot and does not include it as a candidate. No payload reclamation is required because logical membership is determined solely by the validity bit.

If the old bitmap value shows the bit transitioned from 1 to 0, the delete performs bookkeeping and sets $\mathcal{T}[u] \leftarrow$ `INVALID`. If the bit was already 0, clearing it again does not change the logical state. Either way, repeated deletes are safe. □

*3.5.2 Time Complexity.* We analyze the complexity relative to the number of existing vectors $N$ and the dimensionality $D$. For ingestion, the standard dynamic array approach requires $O(N)$ time in the worst case to copy data during capacity resizing. In contrast, the slab-based allocation in SIVF operates in $O(1)$ time, as it involves only pointer manipulation and atomic counter updates, independent of the current list size. For deletion, traditional IVF implementations without a reverse index require scanning the target inverted list, yielding a complexity of $O(N/n_{list})$. Even with a reverse index, physical removal requires $O(N/n_{list})$ data movement to maintain array contiguity. SIVF achieves $O(1)$ deletion complexity by utilizing the Address Translation Table to locate the target slot in constant time and performing a logical invalidation via a single atomic instruction. For search, the complexity remains $O(nprobe \cdot (N/n_{list}) \cdot D)$, which is asymptotically equivalent to standard IVF. While the linked-slab layout introduces pointer-chasing overhead, it does not alter the algorithmic complexity class.

*3.5.3 Space Complexity.* The space complexity of SIVF consists of the vector storage, the slab metadata, and the address translation structures. Let $S$ be the slab capacity (fixed at 32). The metadata overhead per vector is constant, scaling as $O(1/S)$. The Address Translation Table introduces a linear space overhead of $O(N_{max})$, where each entry requires 8 bytes. Comparing this to standard dynamic arrays, which often reserve up to $2\times N$ capacity to amortize resizing costs, SIVF maintains a tighter memory footprint. The external fragmentation is bounded by the size of the free pool, and internal fragmentation is limited to the unused slots in the tail slab of each list. Consequently, the total space complexity is $O(N \cdot D)$, maintaining linear scalability with dataset size.

## 3.6 System Implementation

We implement SIVF by extending the GPU components of the widely-adopted Faiss library [6], integrating our proposed data structures and algorithms into its core indexing architecture. We have open-sourced our complete implementation and evaluation scripts on GitHub.

Implementing SIVF required substantial modifications to the Faiss GPU backend and associated experimental infrastructure. Table 1 summarizes some of the key source files added or modified in the repository based on our development history. In total, the implementation involves approximately 9,000 lines of code changes

**Table 1: Summary of Key Source Files of SIVF Implementation in Faiss [6]**

| *File Path* | *Description* |
|---|---|
| *Core SIVF Architecture & Memory Management* | |
| `faiss/gpu/GpuIndexSIVF.h` | Header definition for the SIVF index class, inheriting from `GpuIndex`. |
| `faiss/gpu/GpuIndexSIVF.cu` | Manages host-device synchronization and overrides standard operations. |
| `faiss/gpu/impl/SlabManager.cuh` | Defines the device-side slab structures, arena pointers, and metadata layout. |
| `faiss/gpu/impl/SlabManager.cu` | Slab-based Dynamic Memory Allocator (SDMA) and memory pool management. |
| `faiss/gpu/impl/SIVFStructs.cuh` | Shared data structures and metadata definitions used across host and device code. |
| *CUDA Kernels (Ingestion, Search, Deletion)* | |
| `faiss/gpu/impl/SIVFAppend.cuh` | Device function templates for the ingestion logic. |
| `faiss/gpu/impl/SIVFAppend.cu` | Lock-free insertion kernel, atomic slot reservation, and slab expansion logic. |
| `faiss/gpu/impl/SIVFSearch.cuh` | Warp-Level Collaborative Search (WLCS) and shared memory reduction. |
| `faiss/gpu/impl/SIVFSearch.cu` | Search kernel, including distance computation and heap aggregation. |
| `faiss/gpu/SIVFDeletion.cu` | Bitmap-based lazy eviction and deletion kernels. |

across more than 45 source files. Core system changes are concentrated in the GPU index implementation, incorporating new GPU-native insertion, deletion, and search operators (e.g., `GpuIndexSIVF`, `SIVFAppend`, `SIVFSearch`, `SIVFDelete`), alongside a custom slab-based memory manager (e.g., `SlabManager`) and lock-free metadata structures (e.g., `SIVFStructs`). Furthermore, we developed a comprehensive benchmarking suite that extends beyond standard metric testing to include end-to-end streaming simulations, memory scalability analysis, and automated visualization scripts, ensuring a rigorous and reproducible evaluation.

Integrating SIVF into Faiss presented unique engineering challenges, particularly in reconciling our dynamic memory model with Faiss's static resource management. Unlike standard Faiss indices which rely on pre-allocated flat arrays (managed via `DeviceTensor`), SIVF introduces a custom `SlabManager` that operates directly on a raw GPU memory arena. To maintain compatibility, we implemented `GpuIndexSIVF` as a direct subclass of `GpuIndex`, overriding the virtual `addImpl` and `searchImpl` methods. This polymorphism allows SIVF to serve as a drop-in replacement in existing Faiss pipelines. Of note, our implementation respects Faiss's `GpuResources` context, ensuring that all SIVF kernels (append, search, delete) are dispatched onto the user-specified CUDA streams. This design allows SIVF to support asynchronous ingest-search pipelines without blocking the default stream, a requirement for high-throughput streaming scenarios. Finally, we extended the SWIG interface files to expose the new `SIVF` class and its configuration parameters (e.g., slab size, relaxation factor) to the Python layer, enabling seamless interaction with PyTorch and NumPy workflows.

## 4 Evaluation

Our evaluation benchmarks SIVF directly against the native Faiss GPU implementations, specifically targeting ingestion throughput and deletion latency under high-concurrency streaming workloads.

[Dongfang: Add multi-GPU results]

### 4.1 Experimental Setup

To ensure a comprehensive evaluation, we adopt a two-tiered baseline strategy. First, our primary baseline is the standard Faiss GPU IVF, which serves as a direct architectural counterpart; this allows us to isolate the performance gains specifically attributable to our proposed memory management innovations, excluding confounding factors from differing algorithmic paradigms. Second, to position SIVF within the broader indexing landscape, we in Section 4.7 compare against representative non-IVF indices, including GPU CAGRA [38] (dynamic graph), GPU Flat (brute-force), HNSW [34] (dynamic graph), NSG [8] (static graph), and LSH [4] (hashing).

All experiments were conducted on a bare-metal node from the Chameleon Cloud testbed [23] (CHI@UC site). The platform is equipped with dual-socket Intel Xeon Gold 6126 CPUs (Skylake microarchitecture), providing a total of 24 physical cores (48 threads with Hyper-Threading) clocked at 2.60 GHz. The host memory consists of 192 GiB of RAM. For acceleration, the node features an NVIDIA Quadro RTX 6000 GPU with 24 GB of GDDR6 VRAM. The system runs on a Ubuntu 24.04 environment with the CUDA toolkit installed: Driver ver. 560.35.05 and CUDA ver. 12.2.

Each reported data point represents the average of at least three independent executions. Error bars are omitted in figures where the standard deviation is negligible to maintain visual clarity. Unless explicitly stated (as in Section 4.4), our experiments utilize synthetic datasets consisting of random 128-dimensional vectors generated from a uniform distribution. We employ this configuration as a neutral baseline to isolate system-level overheads (e.g., memory allocation, PCIe transfer) from data distribution effects. Note that because uniform data lacks the clustering structure inherent in real-world applications, the performance advantages of SIVF in microbenchmarks are primarily architectural and appear more conservative than real-world datasets.

### 4.2 Microbenchmarks

*4.2.1 Ingestion.* We evaluate ingestion throughput against the standard Faiss GPU IVFFlat baseline, varying database size ($N_B$: 1M–4M) and cluster count ($n_{list}$: 1024–16384). As shown in Figure 2, SIVF achieves consistent performance advantages with up to 2.65× speedup.

*Scalability (Fig. 2a).* SIVF maintains a stable throughput of ≈4.2M vec/s as $N_B$ increases, significantly outperforming the baseline (≈1.7M vec/s). This stability validates our lock-free SlabManager, which ensures $O(1)$ insertion cost independent of index occupancy,

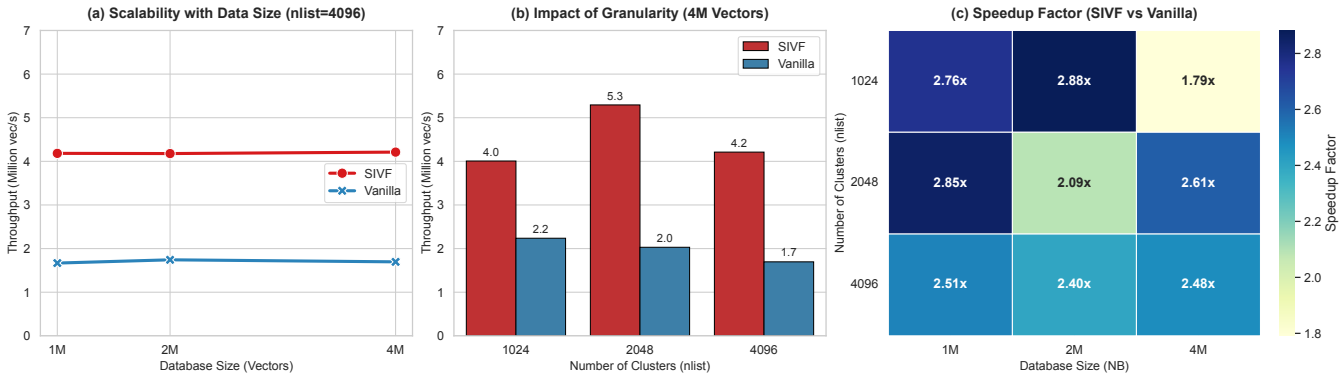


**Figure 2: Microbenchmark performance evaluation of vector ingestion.**

thereby avoiding the contiguous memory resizing overheads inherent to the baseline.

*Granularity Impact (Fig. 2b).* While the baseline throughput degrades monotonically as $n_{list}$ increases (dropping from 2.2M to 1.7M vec/s), SIVF exhibits a performance sweet spot at $n_{list} = 2048$, peaking at 5.3M vec/s. At this optimal granularity, SIVF achieves a 2.65× speedup over the baseline. Even at high $n_{list} = 4096$, where scattered memory writes typically hinder performance, SIVF maintains robust throughput at 4.2M vec/s, sustaining a 2.47× advantage over the baseline's 1.7M vec/s.

*Overall Speedup (Fig. 2c).* SIVF outperforms the baseline across all configurations, with speedups typically ranging from 2.4× to 2.9×. Even under maximum contention (4M vectors, $n_{list} = 1024$), SIVF retains a 1.79× advantage, confirming the efficiency of the non-blocking, slab-based pipeline for streaming scenarios.

*4.2.2 Search.* We evaluate query performance against the Vanilla Faiss GPU baseline. While SIVF trades memory contiguity for mutability, Figure 3 confirms competitive efficiency, reaching up to 91% of baseline performance in optimized configurations.

*Scalability (Fig. 3a).* SIVF throughput scales predictably from ≈383K to 118K QPS as database size grows (100K–500K, $n_{list} = 4096$). The trend closely mirrors the baseline, validating the stability of our slab traversal logic under increasing load.

*Granularity Impact (Fig. 3b).* Increasing $n_{list}$ from 1024 to 4096 boosts SIVF throughput by 69% (70K → 118K QPS), effectively balancing the reduced search space per probe against memory access overhead. However, at finer granularity ($n_{list} = 16384$), performance slightly regresses to 100K QPS. This indicates that while finer clusters reduce distance computations within lists, the overhead of managing excessive lists and reduced memory locality eventually begins to diminish returns.

*Query Efficiency (Fig. 3c).* The efficiency ratio, which is defined as ($QPS_{SIVF}/QPS_{Vanilla}$), highlights the effectiveness of warp-level optimizations. In sparse scenarios ($n_{list} = 16384$, 100K vectors), SIVF achieves 0.91× efficiency, nearly matching contiguous arrays. As lists lengthen (500K vectors), the ratio stabilizes between 0.40× and 0.52×, reflecting the necessary physical trade-off of pointer chasing for dynamic memory support.

*4.2.3 Deletion.* We evaluate the efficiency of the vector deletion mechanism in SIVF by comparing it against the standard Faiss GPU IVF baseline. The experiment measures latency and throughput when removing a batch of 10,000 vectors from a populated index of one million 128-dimensional vectors.

Figure 4 illustrates the performance comparison between the two methods. The results indicate that the baseline Faiss implementation incurs a substantial deletion latency of approximately 202.2 ms. In stark contrast, SIVF completes the same batch deletion operation in an average of 0.68 ms. This represents a massive speedup of 298.5×. Correspondingly, deletion throughput surges from $4.9 \times 10^4$ vec/s in the baseline to nearly $1.5 \times 10^7$ vec/s in SIVF.

## 4.3 Parameter Sensitivity and Ablation Study

We evaluate system robustness by sweeping three parameters: (i) vector capacity factor (*maxvec_factor*, or *mv*), (ii) slab pool redundancy (*slab_factor*, or *sl*), and (iii) deletion batch size (*b*). These control logical headroom, pre-allocated memory for bursts, and the trade-off between amortization and freshness. Figures 5 and 6 summarize the results.

*Impact of Pre-allocation Strategy (Fig. 5a).* Insertion throughput correlates positively with memory provisioning. Increasing *maxvec_factor* from 1.1 to 1.5 yields a clear performance uplift (e.g., 2.71 → 2.95 M vec/s), as larger initial capacities reduce the frequency of dynamic resizing. Similarly, a larger batch size ($b = 8192$) consistently outperforms smaller batches, culminating in a peak throughput of 3.23M vec/s. This confirms that SIVF benefits from amortizing kernel launch overheads via batching and minimizing allocation contention via generous pre-allocation.

*Deletion Stability and Resource Constraints (Fig. 5b).* Deletion performance reveals a tradeoff between resource provisioning and batching efficiency. Under tight memory constraints ($mv : 1.1, sl : 1.0$), throughput is highly sensitive to batch size, dropping to 1.08M vec/s with small batches due to allocation contention. However, relaxing memory constraints by increasing *slab_factor* to 1.3 or

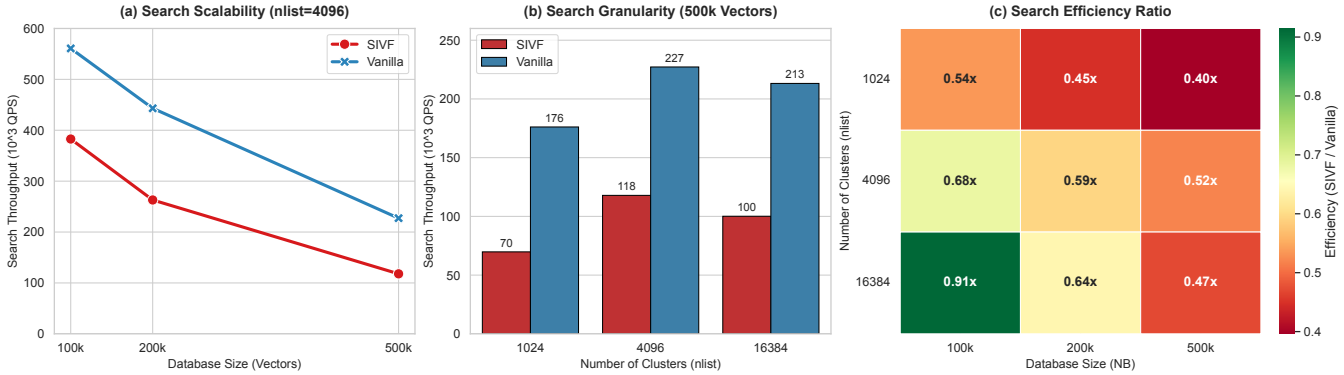


**Figure 3: Microbenchmark performance evaluation of vector search.**

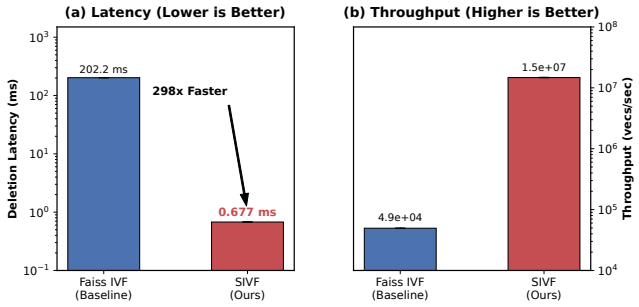


**Figure 4: Microbenchmark performance of vector deletion.**

*maxvec_factor* to 1.5 eliminates this bottleneck, stabilizing throughput around 1.7M vec/s regardless of batch size. This demonstrates that SIVF's slab allocator effectively decouples deletion latency from fragmentation when sufficiently provisioned.

*Latency Stability (Fig. 6).* Figure 6 analyzes the end-to-end latency for index mutations. Insertion latency (Fig. 6a) remains consistent between 3.09 ms and 3.69 ms. Performance correlates with memory provisioning: increasing *maxvec_factor* to 1.5 minimizes dynamic resizing overhead, achieving the lowest insertion latency of 3.09 ms. Deletion operations (Fig. 6b) are significantly faster, maintaining sub-millisecond latency (0.58–0.93 ms) across all settings. This 3×–5× speed advantage over insertion stems from SIVF's lock-free bitmap mechanism, which avoids the data movement required for insertion. Notably, deletion latency benefits from batch amortization, dropping from 0.93 ms to 0.63 ms as batch size increases, confirming that kernel launch overhead dominates the cost of these lightweight bitwise operations.

## 4.4 Real-world Datasets

We benchmark SIVF using four popular real-world datasets representing diverse modalities: Deep1B [50] (96 dimensions, deep features), SIFT1M [22] (128 dimensions, vision), T2I-1B [42] (200 dimensions, multimodal/RAG), and GIST1M [22] (960 dimensions, high-dim features). While the subset size (1M vectors) fits within GPU memory, these experiments validate performance within the *active window* of streaming applications. Section 4.5 will report their stability under high-churn workloads.

*High-Throughput Ingestion.* As shown in Figure 7, SIVF dominates ingestion scalability across all dimensions. On Deep1B, SIVF achieves a peak throughput of 4.38M vec/s (120× speedup over the baseline's 36.4K vec/s). This advantage persists on SIFT1M (3.78M vec/s, 105×) and T2I-1B (2.91M vec/s, 84×). Even on high-dimensional GIST1M, SIVF maintains 852.7K vec/s (36× speedup), confirming that our pre-allocated slab architecture effectively saturates GPU bandwidth by eliminating dynamic resizing overhead.

*Low-Latency Deletion.* Figure 8 highlights the critical performance divergence. Baseline latency scales poorly due to CPU-GPU roundtrips, varying from 1.2s (Deep1B) to 11.8s (GIST1M). In contrast, SIVF's in-place bitmap updates decouple latency from dimension, maintaining sub-millisecond performance ($< 0.9$ ms) across all datasets. Notably on GIST1M, SIVF reduces latency from 11,843 ms to 0.89 ms, a 13,307× improvement, validating its feasibility for real-time streams.

*Search Performance and Trade-offs.* Figure 9 compares query throughput (QPS). SIVF exhibits a lower-dimension advantage: on Deep1B, it outperforms the baseline by 2.07× (59.8K vs. 28.9K QPS) while maintaining comparable recall (91.9% vs 92.0%, not shown in the figure). On SIFT1M, it retains a 1.5× lead. Performance parity is observed on T2I-1B (17.8K vs 18.6K QPS). A trade-off appears on high-dimensional GIST1M, where SIVF achieves 37% of baseline QPS. This results from reduced memory coalescing efficiency when traversing non-contiguous slabs with large vectors, which is a necessary compromise to enable instant, lock-free mutability.

## 4.5 End-to-End Streaming Performance

We evaluate real-time applicability using a *Sliding Window* benchmark that mimics production streams. The system maintains a fixed active window ($W$) by ingesting a new batch ($B$) and evicting the oldest batch ($B$) at each step. We test on SIFT1M ($W = 200$K, $B = 10$K) and GIST1M ($W = 100$K, $B = 5$K).

As shown in Figure 10, the standard Faiss baseline suffers from high latency due to the CPU-GPU roundtrip required for index reconstruction, causing system freezes of 355 ms (SIFT1M) and

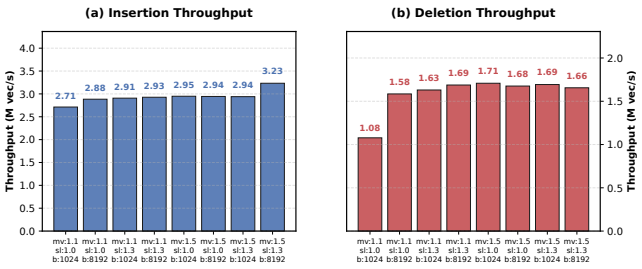


**Figure 5: Throughput sensitivity analysis.**

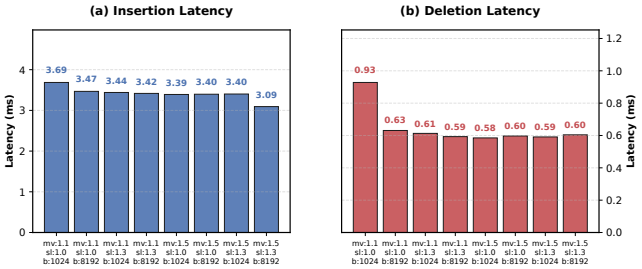


**Figure 6: Latency sensitivity analysis.**

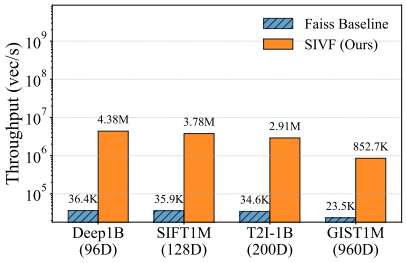


**Figure 7: Ingestion throughput.**

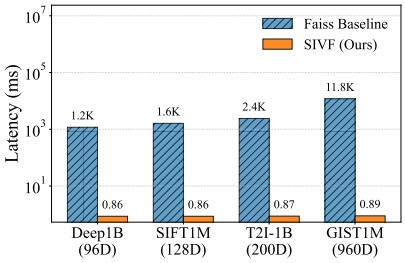


**Figure 8: Deletion latency.**

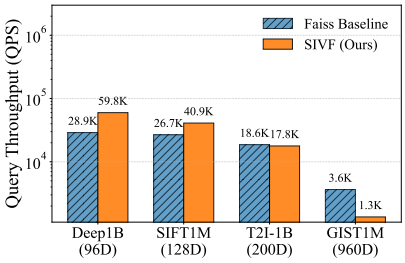


**Figure 9: Search performance (QPS).**

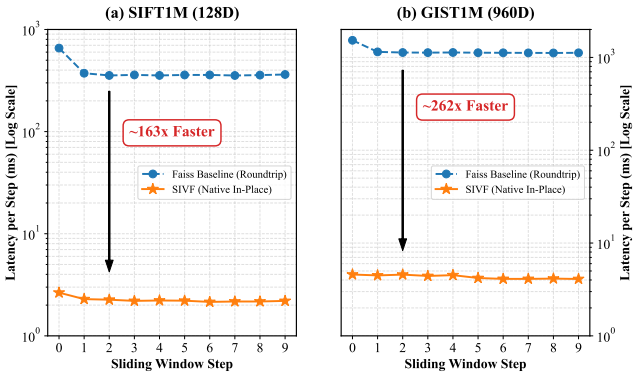


**Figure 10: End-to-End Streaming Performance.**

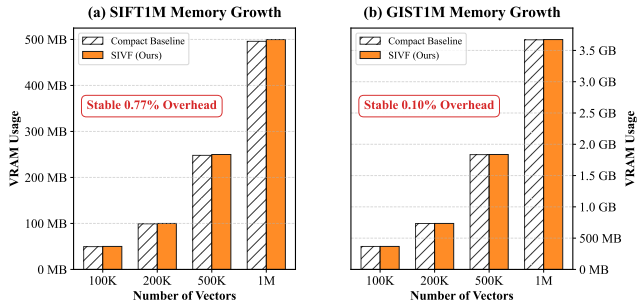


**Figure 11: Memory Efficiency and Scalability.**

> 1.1 s (GIST1M) per update. In contrast, SIVF executes updates strictly in VRAM via slab-based management, reducing per-step latency to ≈ 2.2 ms (SIFT1M) and 4.2 ms (GIST1M). This yields speedups of 163× and 262×, respectively. Notably, the performance gap widens with dimensionality (GIST1M), where the baseline is bottlenecked by PCIe saturation. SIVF eliminates this bottleneck, maintaining single-digit millisecond latency and transforming the GPU IVF index into a real-time streaming engine.

## 4.6 Memory Efficiency and Scalability

A common concern with dynamic graph-based or list-based indices is the potential for memory bloating due to structural overhead, such as pointers and metadata, as well as fragmentation. To quantify the space complexity of SIVF, we conducted a memory footprint analysis comparing the allocated VRAM usage of SIVF against a theoretically compact array baseline across varying dataset sizes ranging from 100K to 1M vectors.

Figure 11 illustrates the memory growth trends for both SIFT1M and GIST1M. The results demonstrate that SIVF exhibits deterministic linear scalability with negligible structural overhead. For SIFT1M ($d = 128$), the overhead stabilizes at 0.77%, while for the high-dimensional GIST1M ($d = 960$), it is merely 0.10%. This efficiency stems from our coarse-grained slab design, where the metadata cost (128-byte header) is amortized over a batch of 32 vectors. Consequently, SIVF achieves orders-of-magnitude performance gains in dynamic operations without imposing significant storage penalties compared to static, compact indices.

Furthermore, we verified the efficacy of our memory reclamation strategy. In a stress test (not shown in the figure) involving the deletion of 50% of the dataset followed by immediate re-insertion, SIVF maintained a constant memory footprint without triggering

**Table 2: Comparison of ingestion throughput (K vec/s) and deletion latency (ms) across different datasets.**

| Method | SIFT1M | | T2I-1B | | GIST1M | |
|---|---|---|---|---|---|---|
| | Add | Del | Add | Del | Add | Del |
| Flat [6] | 9,169 | 836 | 6,020 | 1,160 | 1,230 | 1,143 |
| CAGRA [38] | 305 | 3,030 | 259 | 3,773 | 97.1 | 10,215 |
| HNSW [34] | 25.5 | N/A | 25.7 | N/A | 0.6 | N/A |
| NSG [8] | 3.7 | N/A | 3.2 | N/A | 0.7 | N/A |
| LSH [4] | 787 | 14.6 | 428 | 16.3 | 36.6 | 9.2 |
| **SIVF** (this work) | **3,780** | **0.86** | **2,910** | **0.87** | **853** | **0.89** |

OS-level deallocation. The deletion operation completed in sub-millisecond time per batch (e.g., 4.04 ms for 100K GIST vectors), confirming that memory slots are logically reclaimed via bitmap toggling and immediately available for reuse. This "zero-cost" reclamation ensures that SIVF can sustain long-running streaming workloads without suffering from memory leaks or fragmentation-induced bloat.

## 4.7 Graph-based Index Comparison

To position SIVF within the broader indexing landscape, we benchmarked it against five representative non-IVF architectures: GPU CAGRA [38] (graphs), Faiss GPU Flat (brute-force baseline, i.e., no index), HNSW [34] (dynamic graphs), (3) LSH [4] (legacy hash-based approaches), and (4) NSG [8] (static graphs). Table 2 summarizes ingestion throughput and deletion latency across three datasets: SIFT1M, T2I-1B, and GIST1M.

Table 2 reports the results of vector ingestion and deletion in a streaming context across three datasets. CPU-based graph indices (HNSW, NSG) suffer catastrophic performance degradation on high-dimensional data, dropping to merely 600 vec/s on GIST1M due to cache thrashing, and lack support for native deletion. While GPU Flat achieves high ingestion throughput by bypassing indexing overhead, it incurs prohibitive deletion latencies (> 1s) due to $O(N)$ data compaction and CPU-GPU synchronization. Notably, the state-of-the-art GPU graph index, CAGRA, exhibits severe rigidity in streaming scenarios; enforcing true physical deletion to prevent memory leaks necessitates full index reconstruction, causing latencies to spike to over 10 seconds on GIST1M. In contrast, SIVF emerges as the only architecture enabling holistic stream processing: it sustains GPU-class ingestion rates (e.g., 853K vec/s on GIST1M, 8.8× faster than CAGRA) while providing sub-millisecond physical slot reclamation (≈ 0.89 ms). This effectively bridges the gap between static high-throughput search and dynamic mutability, outperforming graph reconstruction latencies by four orders of magnitude.

# 5 Conclusion

This work addresses a structural limitation inherent in the de facto GPU-accelerated Inverted File (IVF) indices, which effectively precludes their use in streaming vector analytics due to rigid and static memory layouts. A new GPU-native indexing method, namely SIVF, is proposed, which relocates memory management responsibilities entirely onto the device. SIVF has been implemented and integrated into the industry-standard Faiss library; extensive evaluation spanning 6 baseline architectures and 5 datasets demonstrates that SIVF effectively bridge the latency/throughput gap between static high-throughput search and dynamic mutability with negligible memory overhead in streaming vector analytics.

Despite the orders-of-magnitude improvement in mutation latency, current SIVF design entails specific trade-offs. First, as observed in the GIST1M (960 dimensions) evaluation, this results in a 37% reduction in raw search QPS compared to contiguous static indices. While acceptable for write-heavy streaming scenarios, future work will address this by exploring *adaptive super-slabs* that enforce page-aligned storage to restore memory bandwidth utilization. Second, the consistency model of SIVF prioritizes throughput over strict linearizability. While sufficient for the probabilistic nature of approximate nearest neighbor search, future work will investigate lightweight snapshot isolation mechanisms to support transactional semantics without compromising the lock-free ingestion speed.

# References


[1] Chatzakis, M., Papakonstantinou, Y., and Palpanas, T. Darth: Declarative recall through early termination for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 4 (Sept. 2025).

[2] Chu, G., He, Y., Dong, L., Ding, Z., Chen, D., Bai, H., Wang, X., and Hu, C. Efficient algorithm design of optimizing spmv on gpu. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2023), HPDC '23, Association for Computing Machinery, p. 115–128.

[3] Chung, J. W., Lin, H., and Zhao, W. Locality-sensitive indexing for graph-based approximate nearest neighbor search. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval* (New York, NY, USA, 2025), SIGIR '25, Association for Computing Machinery, p. 2418–2428.

[4] Datar, M., Immorlica, N., Indyk, P., and Mirrokni, V. S. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the Twentieth Annual Symposium on Computational Geometry* (New York, NY, USA, 2004), SCG '04, Association for Computing Machinery, p. 253–262.

[5] Deng, L., Chen, P., Zeng, X., Wang, T., Zhao, Y., and Zheng, K. Efficient data-aware distance comparison operations for high-dimensional approximate nearest neighbor search. *Proc. VLDB Endow. 18*, 3 (Nov. 2024), 812–821.

[6] Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.-E., Lomeli, M., Hosseini, L., and Jégou, H. The faiss library.

[7] Engels, J., Landrum, B., Yu, S., Dhulipala, L., and Shun, J. Approximate nearest neighbor search with window filters. In *Forty-first International Conference on Machine Learning* (2024).

[8] Fu, C., Xiang, C., Wang, C., and Cai, D. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proc. VLDB Endow. 12*, 5 (Jan. 2019), 461–474.

[9] Gao, J., Gou, Y., Xu, Y., Yang, Y., Long, C., and Wong, R. C.-W. Practical and asymptotically optimal quantization of high-dimensional vectors in euclidean space for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 3 (June 2025).

[10] Gao, J., and Long, C. Rabitq: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. *Proc. ACM Manag. Data 2*, 3 (May 2024).

[11] Gong, Z., Zeng, Y., and Chen, L. Accelerating approximate nearest neighbor search in hierarchical graphs: Efficient level navigation with shortcuts. *Proc. VLDB Endow. 18*, 10 (June 2025), 3518–3530.

[12] Gou, Y., Gao, J., Xu, Y., and Long, C. Symphonyqg: Towards symphonious integration of quantization and graph for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 1 (Feb. 2025).

[13] Hua, Z., Mo, Q., Yao, Z., Cui, L., Liu, X., Wang, G., Wei, Z., Liu, X., Tang, T., Liu, S., and Qu, L. Dynamically detect and fix hardness for efficient approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[14] Hui, X., Xu, Y., Guo, Z., and Shen, X. Esg: Pipeline-conscious efficient scheduling of dnn workflows on serverless platforms with shareable gpus. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 42–55.

[15] Hui, X., Xu, Y., and Shen, X. Fluidfaas: A dynamic pipelined solution for serverless computing with strong isolation-based gpu sharing. In *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed*

*Computing* (New York, NY, USA, 2025), HPDC '25, Association for Computing Machinery.

[16] Hwang, S., Lee, E., Oh, H., and Yi, Y. Fasop: Fast yet accurate automated search for optimal parallelization of transformers on heterogeneous gpu clusters. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 253–266.

[17] Indyk, P., and Xu, H. Worst-case performance of popular approximate nearest neighbor search implementations: Guarantees and limitations. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023* (2023), A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds.

[18] Jääsaari, E., Hyvönen, V., and Roos, T. Lorann: Low-rank matrix factorization for approximate nearest neighbor search. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024* (2024), A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds.

[19] Jiang, M., Yang, Z., Zhang, F., Hou, G., Shi, J., Zhou, W., Li, F., and Wang, S. Digra: A dynamic graph indexing for approximate nearest neighbor search with range filter. *Proc. ACM Manag. Data 3*, 3 (June 2025).

[20] Johnson, J., Douze, M., and Jégou, H. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data 7*, 3 (2021), 535–547.

[21] Jung, S., Park, Y., Lee, H., Oh, Y. H., and Lee, J. W. Angular distance-guided neighbor selection for graph-based approximate nearest neighbor search. In *Proceedings of the ACM on Web Conference 2025* (New York, NY, USA, 2025), WWW '25, Association for Computing Machinery, p. 4014–4023.

[22] Jégou, H., Douze, M., and Schmid, C. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence 33*, 1 (2011), 117–128.

[23] Keahey, K., Anderson, J., Zhen, Z., Riteau, P., Ruth, P., Stanzione, D., Cevik, M., Colleran, J., Gunawi, H. S., Hammock, C., Mambretti, J., Barnes, A., Halbach, F., Rocha, A., and Stubbs, J. Lessons learned from the chameleon testbed. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association, July 2020.

[24] Laut, S., Borrell, R., and Casas, M. Extending sparse patterns to improve inverse preconditioning on gpu architectures. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 200–213.

[25] Leyba, K., Hofmeyr, S., Forrest, S., Cannon, J., and Moses, M. Simcov-gpu: Accelerating an agent-based model for exascale. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 322–333.

[26] Li, M., Yan, X., Lu, B., Zhang, Y., Cheng, J., and Ma, C. Attribute filtering in approximate nearest neighbor search: An in-depth experimental study. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[27] Li, X., Laguna, I., Fang, B., Swirydowicz, K., Li, A., and Gopalakrishnan, G. Design and evaluation of gpu-fpx: A low-overhead tool for floating-point exception detection in nvidia gpus. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2023), HPDC '23, Association for Computing Machinery, p. 59–71.

[28] Li, Z., Ke, X., Zhu, Y., Yu, B., Zheng, B., and Gao, Y. Scalable graph indexing using gpus for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[29] Li, Z., Ke, X., Zhu, Y., Yu, B., Zheng, B., and Gao, Y. Scalable graph indexing using gpus for approximate nearest neighbor search. In *Proceedings of the 2026 International Conference on Management of Data (SIGMOD)* (2026).

[30] Liang, A., Zhang, P., Yao, B., Chen, Z., Song, Y., and Cheng, G. Unify: Unified index for range filtered approximate nearest neighbors search. *Proc. VLDB Endow. 18*, 4 (Dec. 2024), 1118–1130.

[31] Liu, M., Zhong, S., Yang, Q., Han, Y., Liu, X., and Ma, Y. Webanns: Fast and efficient approximate nearest neighbor search in web browsers. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval* (New York, NY, USA, 2025), SIGIR '25, Association for Computing Machinery, p. 2483–2492.

[32] Lu, D., Feng, S., Zhou, J., Solleza, F., Schwarzkopf, M., and Çetintemel, U. Vectraflow: Integrating vectors into stream processing. In *Proceedings of the 2025 Conference on Innovative Data Systems Research (CIDR)* (Jan. 2025).

[33] Lu, K., Xiao, C., and Ishikawa, Y. Probabilistic routing for graph-based approximate nearest neighbor search. In *Forty-first International Conference on Machine Learning* (2024).

[34] Malkov, Y. A., and Yashunin, D. A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Trans. Pattern Anal. Mach. Intell. 42*, 4 (Apr. 2020), 824–836.

[35] Maurya, A., Rafique, M. M., Tonellot, T., AlSalem, H. J., Cappello, F., and Nicolae, B. Gpu-enabled asynchronous multi-level checkpoint caching and prefetching. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2023), HPDC '23, Association for Computing Machinery, p. 73–85.

[36] Miao, D., Laguna, I., and Rubio-González, C. Floatguard: Efficient whole-program detection of floating-point exceptions in amd gpus. In *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2025), HPDC '25, Association for Computing Machinery.

[37] Mohoney, J., Sarda, D., Tang, M., Chowdhury, S. R., Pacaci, A., Ilyas, I. F., Rekatsinas, T., and Venkataraman, S. Quake: adaptive indexing for vector search. In *Proceedings of the 19th USENIX Conference on Operating Systems Design and Implementation* (USA, 2025), OSDI '25, USENIX Association.

[38] Ootomo, H., Naruse, A., Nolet, C., Wang, R., Feher, T., and Wang, Y. CAGRA: Highly Parallel Graph Construction and Approximate Nearest Neighbor Search for GPUs . In *2024 IEEE 40th International Conference on Data Engineering (ICDE)* (Los Alamitos, CA, USA, May 2024), IEEE Computer Society, pp. 4236–4247.

[39] Peng, Z., Qiao, M., Zhou, W., Li, F., and Deng, D. Dynamic range-filtering approximate nearest neighbor search. *Proc. VLDB Endow. 18*, 10 (June 2025), 3256–3268.

[40] Peng, Z., Thomadakis, P., Pienaar, J., and Kestor, G. Liteform: Lightweight and automatic format composition for sparse matrix-matrix multiplication on gpus. In *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2025), HPDC '25, Association for Computing Machinery.

[41] Qiu, R., and Tang, J. Efficient approximate nearest neighbor search via hemisphere centroids graph. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[42] simhadri, H. V., Aumüller, M., Douze, M., Baranchuk, D., Ingber, A., Liberty, E., Williams, G., Landrum, B., Manohar, M. D., Karjikar, M., Dhulipala, L., Chen, M., Chen, Y., Ma, R., Zhang, K., Cai, Y., Shi, J., Zheng, W., Chen, Y., Yin, J., and Huang, B. Results of the big ANN: NeurIPS'23 competition. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track* (2025).

[43] Sun, P., Simcha, D., Dopson, D., Guo, R., and Kumar, S. SOAR: improved indexing for approximate nearest neighbor search. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023* (2023), A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds.

[44] Tran, A., Laguna, I., and Gopalakrishnan, G. Fpboxer: Efficient input-generation for targeting floating-point exceptions in gpu programs. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 83–93.

[45] Voruganti, S., and Özsu, M. T. Mirage-anns: Mixed approach graph-based indexing for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 3 (June 2025).

[46] Wang, M., Lv, L., Xu, X., Wang, Y., Yue, Q., and Ni, J. An efficient and robust framework for approximate nearest neighbor search with attribute constraint. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023* (2023), A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds.

[47] Wang, Z., Zhang, J., and Hu, W. Wow: A window-to-window incremental index for range-filtering approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[48] Wei, J., Lee, X., Liao, Z., Palpanas, T., and Peng, B. Subspace collision: An efficient and accurate framework for high-dimensional approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 1 (Feb. 2025).

[49] Xu, Q., Zhang, F., Li, C., Cao, L., Chen, Z., Zhai, J., and Du, X. Harmony: A scalable distributed vector database for high-throughput approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 4 (Sept. 2025).

[50] Yandex, A. B., and Lempitsky, V. Efficient indexing of billion-scale datasets of deep descriptors. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2016), pp. 2055–2063.

[51] Yang, M., Cai, Y., and Zheng, W. CSPG: crossing sparse proximity graphs for approximate nearest neighbor search. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024* (2024), A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds.

[52] Zhang, B., Tian, J., Di, S., Yu, X., Feng, Y., Liang, X., Tao, D., and Cappello, F. Fz-gpu: A fast and high-ratio lossy compressor for scientific computing applications on gpus. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2023), HPDC '23, Association for Computing Machinery, p. 129–142.

[53] Zhang, F., Jiang, M., Hou, G., Shi, J., Fan, H., Zhou, W., Li, F., and Wang, S. Efficient dynamic indexing for range filtered approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 3 (June 2025).

[54] Zhang, Z., Liu, F., Huang, G., Liu, X., and Jin, X. Fast vector query processing for large datasets beyond GPU memory with reordered pipelining. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)* (Santa Clara, CA, Apr. 2024), USENIX Association, pp. 23–40.

[55] Zhong, X., Li, H., Jin, J., Yang, M., Chu, D., Wang, X., Shen, Z., Jia, W., Gu, G., Xie, Y., Lin, X., Shen, H. T., Song, J., and Cheng, P. Vsag: An optimized search framework for graph-based approximate nearest neighbor search. *Proc. VLDB Endow. 18*, 12 (Aug. 2025), 5017–5030.